



Aprendizaje Autónomo 1

Investigación y Comparación de Frameworks

Estudiante: Cristina Isabel Zambrano Cedeño

Tutor: Jonathan Eduardo Tito Ontaneda

Materia: Diseño de Pruebas, Control
de Calidad y Mantenimiento

MAYO 2026

Parte 1: Investigación y Comparación de Frameworks

¿Qué es un framework de pruebas unitarias?

Un framework de pruebas unitarias es una plataforma o conjunto de herramientas de software diseñadas para automatizar el proceso de verificar que los componentes más pequeños y aislados de una aplicación (las "unidades", como funciones, métodos o clases) se comporten exactamente como se espera. Proporciona una estructura estandarizada para escribir casos de prueba, un motor para ejecutarlos de manera masiva y un sistema de reportes para visualizar qué pruebas pasaron y cuáles fallaron.

¿Cómo lo usa un desarrollador?

El flujo de trabajo típico de un desarrollador al utilizar un framework incluye los siguientes pasos:

1. **Configuración:** Instala el framework en el entorno de desarrollo mediante un gestor de paquetes (como npm, pip o maven).
2. **Estructuración:** Crea archivos de prueba dedicados (por ejemplo, `calculadora.test.js`).
3. **Escritura (AAA Pattern):** Escribe funciones de prueba que siguen el patrón *Arrange-Act-Assert* (Organizar el escenario, Actuar ejecutando la función real, y Asegurar/Verificar que el resultado coincide con el esperado).
4. **Ejecución:** Corre un comando en la terminal (ej. `npm test`) para lanzar el motor del framework.
5. **Refactorización:** Si una prueba falla, el desarrollador corrige el código fuente hasta que el reporte visual se muestre en verde.

¿Qué beneficios ofrece?

- **Prevención de regresiones:** Permite modificar el código, es decir refactorizar, con la certeza de que las funcionalidades existentes no se han roto.

- **Documentación viva:** Los archivos de prueba sirven como ejemplos reales y actualizados de cómo debe utilizarse el código.
- **Reducción de costos de depuración:** Encontrar un error en una función aislada mediante una prueba unitaria toma segundos, mientras que detectarlo en producción puede requerir horas de análisis.

Promueve el diseño desacoplado: Para que el código sea fácil de probar unitariamente, debe estar modularizado y seguir buenas prácticas de diseño de software.

Comparación de Frameworks: Jasmine vs. Jest

Para esta comparativa se ha seleccionado **Jest** como el segundo framework, dado que es uno de los más adoptados en el desarrollo moderno y comparte raíces históricas con Jasmine.

Criterio	Jasmine	Jest
Enfoque de Diseño	"Behavior-Driven Development" (BDD). Es un framework independiente (<i>batteries-included</i>).	Desarrollado por Meta (Facebook). Originalmente basado en Jasmine, evolucionó hacia un entorno "todo en uno" optimizado.
Configuración	Requiere configuración manual o herramientas adicionales para el manejo de DOM virtual y mocking avanzado.	"Zero-config". Viene listo para usar con simulaciones, aserciones y cobertura de código integradas por defecto.

Criterio	Jasmine	Jest
Velocidad de Ejecución	Ejecución secuencial estándar de pruebas.	Muy rápido. Ejecuta pruebas en paralelo utilizando <i>workers</i> independientes y prioriza los archivos modificados.

Capacidades Comunes

- **Sintaxis BDD idéntica:** Ambos utilizan la misma estructura semántica basada en bloques describe() para agrupar suites de pruebas e it() o test() para definir los casos individuales.
- **Aserciones (*Matchers*):** Ambos comparten palabras clave intuitivas para verificar resultados, tales como expect(valor).toBe(esperado) o toEqual().
- **Ciclos de Vida:** Soportan ganchos de configuración comunes como beforeEach(), afterEach(), beforeAll() y afterAll() para preparar y limpiar entornos de prueba.

Diferencias Destacables

- **Mocking Integrado:** Mientras que Jasmine utiliza un sistema basado en spyOn(), Jest ofrece un sistema de simulación de módulos completo (jest.mock()) que permite interceptar dependencias de archivos enteros o llamadas a redes de forma nativa sin librerías externas.
- **Pruebas de Instantánea (*Snapshot Testing*):** Jest cuenta con la capacidad de capturar el estado de un componente u objeto y guardarlo en un archivo de texto para comparar cambios futuros de forma automática; Jasmine carece de esta funcionalidad de manera nativa.

Generación de Cobertura: Jest integra internamente la herramienta *Istanbul* para generar reportes de cobertura de código (*code coverage*) directo en la

consola de comandos simplemente agregando el flag `--coverage`. En Jasmine, se deben integrar librerías externas para lograrlo.

Parte 2: Implementación Práctica

Código del algoritmo a probar

Este código está diseñado para buscar un elemento en un arreglo previamente ordenado bajo una complejidad temporal de $O(\log n)$.

```
def busqueda_binaria(arr, target):
    izquierda = 0
    derecha = len(arr) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2 # '/' asegura división
        entera

        if arr[medio] == target:
            return medio # Elemento encontrado, retorna el índice

        if arr[medio] < target:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1 # Elemento no encontrado
```

```
import unittest
from busqueda_binaria import busqueda_binaria

class TestBusquedaBinaria(unittest.TestCase):

    def test_elemento_en_el_medio(self):
        datos = [2, 4, 6, 8, 10]
        self.assertEqual(busqueda_binaria(datos, 6), 2)

    def test_limite_inferior(self):
        datos = [1, 3, 5, 7, 9, 11]
        self.assertEqual(busqueda_binaria(datos, 1), 0)

    def test_limite_superior(self):
        datos = [5, 10, 15, 20, 25]
        self.assertEqual(busqueda_binaria(datos, 25), 4)
```

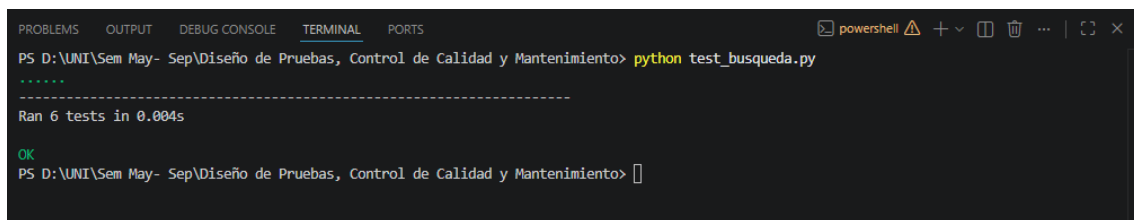
```
def test_elemento_menor_no_existente(self):
    datos = [10, 20, 30, 40]
    self.assertEqual(busqueda_binaria(datos, 5), -1)

def test_elemento_mayor_no_existente(self):
    datos = [10, 20, 30, 40]
    self.assertEqual(busqueda_binaria(datos, 50), -1)

def test_arreglo_vacio(self):
    datos = []
    self.assertEqual(busqueda_binaria(datos, 10), -1)

if __name__ == '__main__':
    unittest.main()
```

Resultado de la ejecución



The screenshot shows a PowerShell terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\UNI\Sem May- Sep\Diseño de Pruebas, Control de Calidad y Mantenimiento> python test_busqueda.py
.....
Ran 6 tests in 0.004s

OK
PS D:\UNI\Sem May- Sep\Diseño de Pruebas, Control de Calidad y Mantenimiento> 
```

Bibliografía

- **Fowler, M.** (2019). *The Practical Test Pyramid*. martinowler.com. Recuperado de <https://martinfowler.com/articles/practical-test-pyramid.html>
- **Jasmine Documentation.** (2024). *Jasmine Core Concepts & BDD Structure*. jasmine.github.io. Recuperado de <https://jasmine.github.io/>
- **Jest Community.** (2025). *Jest: Delightful JavaScript Testing*. jestjs.io. Recuperado de <https://jestjs.io/>
- **Myers, G. J., Sandler, C., & Badgett, T.** (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.

Bibliografía

- **Fowler, M.** (2019). *The Practical Test Pyramid*. martinowler.com. Recuperado de <https://martinfowler.com/articles/practical-test-pyramid.html>
- **Jasmine Documentation.** (2024). *Jasmine Core Concepts & BDD Structure*. jasmine.github.io. Recuperado de <https://jasmine.github.io/>
- **Jest Community.** (2025). *Jest: Delightful JavaScript Testing*. jestjs.io. Recuperado de <https://jestjs.io/>

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.